from DTrace. A probe is a location or activity to which the DTrace framework can bind a request to perform actions, such as logging the system calls, the function calls in user level processes, recording a stack trace and so on. A probe is said to fire when the activity, as specified by the probe description, takes place. When a probe is fired, the requested action will take place. A probe has the following attributes that identify it uniquely:

- It has a name and a unique integer identifier
- It is made available by a provider
- It identifies the module and the function that it instruments

The current probes available on your system can be displayed by *pfexec dtrace -l*. By using various switches, it is also possible to display only the probes belonging to, say, a particular module. For example, *pfexec dtrace -l m:mysql** will list all the probes available via the *mysql** provider. (Note the * in *mysql** denotes all modules with names starting with mysql.)

DTrace probes are implemented by kernel modules called providers, each of which perform a particular kind of instrumentation to create probes. Providers can thus be described as publishers of probes that can be used by DTrace consumers. Providers can be used for instrumenting kernel and user-level code. For user-level code, there are two ways in which probes can be defined: User-Level Statically Defined Tracing (USDT) or the PID provider.

In USDT, custom probe points are inserted into application code according to well-defined guidelines and practices. Refer to *www.solarisinternals.com/wiki/index.php/DTrace_Topics_USDT* for more details. Once the custom probe points are integrated, the application code is compiled and the binary is run, the probes become available for consumption by DTrace user-level consumers. However, unless they are used, the probes have a zero impact on the performance of the application or the system as a whole.

Does this mean that DTrace cannot be used with applications with no USDT probes defined? No, it doesn't.

The PID provider can be used to probe any user-level process, whether USDT probes were defined for it or not. Using the PID provider is a very generic and easy way to play around with DTrace. Code a simple application in your favourite programming language and have fun with DTrace by observing the function call flow, stack trace and a lot more. In the later part of this article, we shall use both of the above for DTracing a running MySQL server.

***Probe descriptions*:** A DTrace probe, as mentioned earlier, is uniquely specified by a 4-tuple, and usually takes the following form:

```
provider:module:function:name
```

If one or more of the fields are missing, the specified fields are interpreted in a right-to-left fashion, i.e., if a probe description is given as *foo:bar*, the probe description matches all probes with the function *foo* and the name *bar*, regardless of the provider or the module. To obtain the desired results, specify all the required fields. You may also want to match all the probes published by any given provider, for which you would use the probe description like *fbt:::*, which matches all the probes of the *fbt* provider. [You can read the manual page of *fbt* at *docs.sun.com/app/docs/doc/816-5177/6mbbc4g4t?a=view*.]

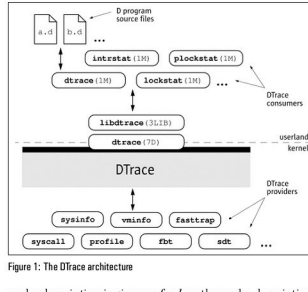

Figure 1: The DTrace architecture

A DTrace consumer is any process that interacts with the DTrace framework. The consumer specifies the instrumentation points by specifying probe descriptions. dtrace is the primary consumer of the DTrace framework. (Now, do you see the difference between DTrace and dtrace?)

DTrace scripts or D-scripts

Programs or scripts to interact with the DTrace framework are written in the D programming language. A D program source file consists of one or more probe clauses that describe the points of instrumentation to be enabled by DTrace. Each probe clause has the following form:

```
probe descriptions
/ predicate /
{
  action statements
}
```

A D program can consist of one or more such probe clauses. The predicate and the list of action statements are optional and may not be required in some scenarios. D programs are described in detail at *docs.sun.com/app/docs/doc/817-6223/chp-prog?a=view*. A D program can be executed by specifying it via the '-s' switch to *dtrace* or making it an executable (like a shell script) and setting 'dtrace' as the interpreter, by putting *#!/usr/sbin/dtrace* in the script.